

ThomsonLint — Future Direction & Roadmap (v2 Vision)

Status: Draft / discussion **Date:** 2026-05-28 **Scope:** Forward-looking architecture. Describes a possible v2 in which the ULP export step is retired in favor of direct, live queries into Fusion Electronics through the MCP server — *conditional* on a set of capabilities being added to that server. This document is a vision and a dependency list, not a committed plan.

1. Executive Summary

ThomsonLint v1 works by **extracting** a design out of Fusion Electronics with a set of ULPs (connectivity/placement JSON, layer-stack JSON, high-DPI image renders), then reviewing those static files against the ontology. The ULP exists because, at the time the project was written, Fusion did not expose a complete programmatic API into the electronics model, so the data had to be dumped to disk first.

As the **Fusion Electronics MCP server** matures, that constraint is lifting. The v2 vision is to **eliminate the ULP and the export step entirely** and have the reviewing LLM query the live design directly — placements, routing, copper pours, layers, and design-rule state — pulling exactly the data each rule needs, when it needs it.

This is achievable, but it is **gated on the MCP server gaining several capabilities that today only the ULP provides**. Those capabilities are enumerated in §5 and are the critical path for v2. Without them, "drop the ULP" would mean streaming huge volumes of raw geometry to the model — slower, more expensive in tokens, and less reliable than the current export. The point of v2 is not just to move the data source; it is to move the *data reduction* into the server so the model reasons over compact, authoritative summaries.

2. Where We Are (v1)

The ULP plays **two distinct roles** today. Separating them is the key to the whole roadmap:

1. **Data access** — walking Fusion's electronics object model to read parts, nets, pins, placements, traces, vias, pours, layers, and holes.
2. **Data reduction / pre-computation** — the ULP does not merely dump raw objects. It computes a derived analysis layer that the review depends on: - Component **type** classification (resistor / capacitor / IC / TVS / zener / connector / ...). - Net **signal classification** (`is_power`, `is_ground`, `is_clock`, `is_differential`, `diff_pair_partner`, `voltage_guess`). - Per-signal **routing aggregates** (`trace_length_mm`, `min_width_mm`, `max_width_mm`, `via_count`, `segment_count`) summed over every wire segment. - **Geometric analyses** (`decoupling_proximity`, `component_edge_distances`, `ground_plane_layers`, `floating_inputs`, `single_pin_nets`). - **Visual evidence** — schematic sheets and per-copper-layer / silkscreen renders at high DPI.

A complete API eliminates role (1) cleanly. Role (2) does not disappear — it has to **move** somewhere. The central design decision for v2 is *where*: into the MCP server (preferred for the deterministic, high-volume parts) or into the agent (appropriate for judgment-based classification). See §5.3.

3. The v2 Vision: Live Design Intelligence

Retiring the export does more than swap an importer. It changes what ThomsonLint *is* — from a batch ETL pipeline that reviews a frozen snapshot into an agent reasoning over live design state. Three shifts follow:

3.1 Snapshot → Live

The review targets the *current* design, not a frozen file. The review becomes iterative and conversational: find an issue → the user fixes it → re-check the same live board in the same session. "Design review" becomes "design-review copilot."

3.2 Eager dump → Lazy, rule-driven pull

Today everything is exported up front, whether a given rule needs it or not. Live querying lets the agent fetch only what the rule under evaluation requires — e.g. pull the switch-node geometry of an SMPS *only* while evaluating the buck/boost layout rules. This is what makes review of large boards tractable instead of a monolithic upfront dump, and it is the primary token-economy lever (§5.3).

3.3 Read → Read/Write (remediation)

A fully functional API is not read-only. Once writes are exposed, ThomsonLint can graduate from *reporting* problems to *proposing and applying fixes* — staging a missing decoupling cap, widening an undersized power trace, adding ground-stitch vias — always behind explicit user confirmation. This is the most valuable long-term destination and should shape the v2 interfaces even if remediation ships later.

3.4 Durable core, swappable adapter

What does **not** change across v1 → v2 is the real intellectual property: `ontology/ontology.json`, the knowledge base, the findings schema, and the rule reasoning. v2 should formalize a clean boundary — a **DesignSource adapter** — behind which the rule engine never knows whether it is talking to a ULP export file, the live MCP server, or a future cloud API. "Same brain, new sensory-motor system."

4. What Must NOT Be Lost

Three properties of the v1 export are easy to lose in a naive "just query live" rewrite and must be preserved by design:

- **Provenance / reproducibility.** The export file is a frozen, version-control-able record of exactly what was reviewed. v2 should **cache its live queries into the same** `-thomson-export-*.json` **contract as a byproduct**, so the export *format* survives as a provenance/cache artifact even though no ULP writes it. This yields live interaction *and* an auditable snapshot.
- **Determinism of analysis.** Moving aggregation from ULP code to the LLM trades reproducibility for flexibility. Deterministic math (trace length, proximity, width extremes) must stay deterministic — computed server-side or by a thin helper the agent calls — never reconstructed by the model summing rows by hand.

- **Offline / headless review.** A live API tethers review to a running, focused Fusion instance. Batch and CI use still need either a serialized snapshot or a cloud/headless read path (§5.6).
-

5. Required Fusion Electronics MCP Server Enhancements

This section is the gating dependency for v2. The copper-pour geometry, full trace routing, layer stackup, and image renders that the ULP produces today are genuinely required for a comprehensive review. For the ULP to be retired, **these capabilities must be incorporated into the Fusion Electronics MCP server itself.** Equally important, the server must provide them in a *reduced / aggregated* form, not only as raw entities — otherwise the token and round-trip cost of replacing the ULP is prohibitive (§5.3).

5.1 Why this is the critical path

The MCP server today exposes the raw electronics object model (parts, nets, pins, elements, pads, wires, vias, pours, layers, holes, and ERC/DRC errors). That covers the ULP's *data-access* role. It does **not** cover the ULP's *data-reduction* role, and it has operational limits (active-editor coupling, dialog blocking, unit drift, no rendering, no headless access). v2 viability depends on closing these gaps.

5.2 Capability requirements

Legend: **Available** = usable today · **Partial** = raw data present but no reduction/aggregate · **Missing** = not available via MCP today.

#	Capability needed for v2	ULP provides today	MCP status today	Required MCP enhancement
1	Component placement (x/y/rotation/side)	Yes	Available (Element)	—
2	Pad locations (absolute board coords)	Yes	Available (Smd / Pad)	—
3	Schematic connectivity (nets, pins, directions)	Yes	Available / Partial (Net , Pin , PinRef)	Confirm pin direction (PAS/IO/PWR/IN/OUT) is exposed on every pin
4	Board outline / area bounding box	Yes	Partial (Board returns id/name only)	Expose board outline geometry + computed width/height/bbox
5	Per-signal routing aggregates (trace_length_mm , min/max_width_mm , via_count , segment_count)	Yes (computed)	Partial (raw Wire / Via only)	Add server-side aggregate query per signal. This is the single highest-value addition — see §5.3
6	Full trace/route geometry per net (segments, width, layer, curve)	Yes (for high-speed nets)	Partial (Wire)	Allow filtered retrieval by signal/layer so only nets under review are pulled
7	Copper pour / polygon geometry (signal, layer, outline, fill, thermals, isolate, rank, voids)	Yes	Partial (PolyPour / PolyShape / PolyCutout presence; full shape/area unclear)	Expose complete pour outlines, computed filled area, void/island data, and clearance — needed for return-path and pour-integrity rules
8	Layer stackup (copper ordering, dielectric/copper thickness, used-vs-unused)	Yes (stackup ULP)	Partial / Missing (Layer table; physical stack ordering + thickness unclear)	Expose the physical stackup: ordered copper layers, thicknesses, prepreg/core, and the used-layer set
9	Geometric analyses (decoupling proximity, component-to-edge distance, nearest-neighbor)	Yes (computed)	Missing (raw coords only)	Either add analysis endpoints, or guarantee enough geometry that a deterministic helper can compute them cheaply
10	DRC / ERC results (violations with type, location, net/part)	No (not in export today)	Available (Error)	Surface and document Error for review consumption — <i>new evidence v1 never had</i>

#	Capability needed for v2	ULP provides today	MCP status today	Required MCP enhancement
11	Rendered images (schematic sheets; per-copper-layer, silkscreen) at high DPI	Yes (image ULP)	Missing (<code>screenshot</code> is viewport-only)	Add a render endpoint: specify document/sheet or board layer-set, DPI, and bounds; return PNG. See §5.4
12	Net class / design rules (width, clearance per class)	Yes	Available / Partial (<code>NetClass</code>)	Confirm width/clearance values are exposed per class
13	Component attributes (MPN, LCSC, DNP/populate, datasheet URL)	Yes	Available (<code>Attribute</code>)	—

5.3 Token economics — why server-side reduction is mandatory, not optional

A core constraint motivates this section: the ULP's pre-computation **lowers the token cost** of a review. A `comet`-sized board already has hundreds of wire segments; a real product board has tens of thousands. If v2 retires the ULP but the MCP only offers *raw* `Wire` / `PolyPour` rows, then computing something as simple as "total trace length and via count for the 5V0 net" forces the agent to page through and sum thousands of rows — many round trips, large context, and arithmetic the model should not be doing by hand.

Therefore the MCP enhancements in §5.2 must include **aggregate/summary queries** (items 5, 7, 8, 9), not just raw-entity reads. The design principle:

Push deterministic reduction to the server. Reserve the LLM for judgment.

A per-signal summary row (item 5) replaces dozens-to-hundreds of raw segments with a single compact object — the same reduction the ULP does today, but served live. This is what makes "drop the export" cheaper than the export, rather than more expensive.

5.4 Rendering / visual evidence

Some checks are irreducibly visual (silkscreen legibility, pour slivers, overall placement gestalt). v1 serves these with high-DPI per-layer PNGs from the image ULP. The MCP `screenshot` query captures only the active viewport, which is not a substitute.

Two-part position: - **Much of today's visual inspection becomes numeric** once full pour and trace geometry (items 6–7) are queryable — "look at the copper" turns into "query the polygon and compute," which is cheaper and more precise. - **What remains genuinely visual requires a render endpoint** (item 11): given a layer-set (or schematic sheet), DPI, and bounds, return a PNG. With that, images become an *on-demand render call*, not a pre-export — and the image ULP can be retired along with the rest.

5.5 Operational reliability

Independent of data coverage, three behaviors observed in practice block a smooth live workflow and must be addressed:

- **Active-editor coupling.** Reads return data only for the currently focused editor; schematic and board are mutually exclusive, and focus can silently flip. v2 needs to query a design's schematic *and* board without UI focus juggling (e.g. address documents/sections explicitly).
- **Dialog blocking.** An open modal dialog (e.g. Grid) blocks reads — the electronics read *times out* with no actionable message. Reads should be serviceable regardless of dialog state (and at minimum fail fast with a clear error).
- **Unit drift.** Reported coordinate units follow the active document and can change mid-session (inch ↔ mm). v2 needs explicit, request-specified units, or a guaranteed-stable unit, so the adapter is not parsing a moving target.

(These three are filed/queued as MCP feature-requests and bug-reports; this roadmap depends on them.)

5.6 Headless / cloud access

For batch and CI review (and to decouple from a running desktop session), the ideal end state is **read-only access to a design in the Autodesk cloud without a live, focused Fusion instance**. This is a larger ask than entity-level reads and is likely longer-horizon, but it is what fully frees v2 from the desktop. Until it exists, the **snapshot-as-cache** (§4) remains the batch/offline path.

6. What Survives v2

- `ontology/ontology.json`, `examples/examples.json`, the knowledge base — the rule corpus is source-agnostic and carries forward unchanged.
- `tests/findings_schema.json`, `tools/validate_findings.py`, `tools/gen_report.py` — the findings → validation → HTML report pipeline is downstream of data acquisition and is unaffected.
- The `-thomson-export-*.json` **format** survives as the provenance/cache artifact (§4), even though it is no longer written by a ULP.

7. Migration Path

Evolutionary, not a rewrite. Each step reuses the proven contract.

- **v1 (today):** ULP export → static files → review. Unchanged.
- **v1.5 (hybrid, low risk):**
 - Introduce the **DesignSource adapter** boundary (§3.4).
 - Add an **MCP-driven exporter** that emits the *exact same* `-thomson-export-*.json` contract by querying the live design — so nothing downstream changes. Requires MCP items 1–4, 12–13, and ideally 5.
 - Wire **DRC/ERC Error (item 10)** into findings as new evidence — pure upside, available now.
 - ULP remains the source for **images (item 11)** and **stackup (item 8)** until those MCP capabilities land.

- **v2 (live):**
- Live, lazy, rule-driven querying via the adapter; ULP fully retired once MCP items 5–11 are available (especially aggregates and rendering).
- Snapshot-as-cache written automatically for provenance and batch.
- Optional, confirmation-gated **remediation** (§3.3).

A useful early validation: generate a `-brd.json` from the live board via MCP and **diff it against the ULP output** for the same design. Byte-for-byte parity on the data fields is the acceptance test for retiring the connectivity/placement export.

8. Risks & Mitigations

Risk	Mitigation
MCP only ever exposes raw entities, not aggregates	Lobby for the aggregate/analysis queries (§5.2 items 5,7,8,9); until then keep the ULP for those fields
Token blowup from streaming raw geometry	Server-side reduction (§5.3); lazy rule-driven pull (§3.2)
Loss of reproducible review record	Snapshot-as-cache in the existing JSON format (§4)
LLM doing deterministic math unreliably	Keep arithmetic in server/helper, model does judgment only
Live workflow fragility (focus, dialogs, units)	§5.5 operational fixes are prerequisites, not nice-to-haves
No headless/CI path	Snapshot-as-cache now; cloud read access long-term (§5.6)
Write/remediation causing unintended design changes	All writes behind explicit user confirmation; dry-run/preview first

9. Open Questions

1. Should the routing/pour/proximity **reduction live in the MCP server or in a thin ThomsonLint-side deterministic helper** fed by raw MCP geometry? (Server is better for tokens and reuse; helper is faster to ship and not blocked on Autodesk.)
2. What is the **minimum aggregate API** that makes the MCP path cheaper than the ULP for a representative production board? (Defines the v1.5→v2 cutover bar.)
3. How should the adapter represent a design that exists **only in the cloud** vs. **only open in desktop** — one interface, two backends, or three?
4. For remediation: which rule classes are **safe to auto-apply** vs. **propose- only**?

10. Asks to Autodesk (MCP server)

Consolidated from §5, in rough priority order:

1. **Per-signal routing aggregates** (trace length, min/max width, via/segment counts) — the highest-leverage addition for both completeness and token cost.
 2. **Copper pour geometry** (outlines, filled area, voids, thermals, isolation).
 3. **Layer stackup** (ordered copper layers, thicknesses, used-layer set).
 4. **Render endpoint** (per-layer / per-sheet PNG at specified DPI and bounds).
 5. **DRC/ERC Error access** — document and stabilize for review consumption.
 6. **Geometric analysis** endpoints (or sufficient geometry for cheap client-side computation).
 7. **Operational:** dialog-independent reads, active-editor-independent (whole- design) reads, explicit/stable coordinate units.
 8. **Headless / cloud read access** (longer-horizon).
-

Authoring note

Drafted by Claude (Anthropic's Claude Code) during a working session that exercised the Fusion Electronics MCP server against the live `comet` design and read the ThomsonLint v1 pipeline end-to-end. Intended as a starting point for discussion, not a committed plan.